

04_shap_diagnosis

June 24, 2026

1 Step 4: SHAP-Based Transfer-Failure Diagnosis (Authoritative v4)

This notebook restores the broader SHAP diagnosis surface while keeping the authoritative v4 framing explicit: Step 4 is a **transfer-diagnosis** layer, not a causal interpretation layer. The methodological repair matters here because the SHAP backgrounds are now source-conditioned and legally reused only within the allowed source-specific scope.

```
[1]: import json
import sys
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from IPython.display import display
from scipy import stats

def find_repo_root() -> Path:
    candidates = [Path.cwd().resolve(), *Path.cwd().resolve().parents]
    for base in candidates:
        if (base / 'data' / 'processed' / 'v4' / 'v4_protocol_manifest.json').
            exists():
            return base
    raise RuntimeError('Could not locate repository root from the current_
working directory.')

REPO_ROOT = find_repo_root()
if str(REPO_ROOT) not in sys.path:
    sys.path.insert(0, str(REPO_ROOT))
if str(REPO_ROOT / 'src') not in sys.path:
    sys.path.insert(0, str(REPO_ROOT / 'src'))

from src.features import ALL_FEATURE_COLS
```

```

STEP_TAG = 'step4'
FIG_PDF = REPO_ROOT / 'report' / 'figures' / 'v4' / 'pdf' / STEP_TAG
FIG_PNG = REPO_ROOT / 'report' / 'figures' / 'v4' / 'png' / STEP_TAG
TABLE_DIR = REPO_ROOT / 'report' / 'tables' / 'v4'
for directory in (FIG_PDF, FIG_PNG, TABLE_DIR):
    directory.mkdir(parents=True, exist_ok=True)

plt.rcParams.update({
    'font.family': 'DejaVu Serif',
    'font.size': 9,
    'axes.labelsize': 9,
    'axes.titlesize': 9,
    'xtick.labelsize': 8,
    'ytick.labelsize': 8,
    'legend.fontsize': 7,
    'figure.dpi': 180,
    'savefig.bbox': 'tight',
    'pdf.fonttype': 42,
    'ps.fonttype': 42,
})

IEEE_ONE_COL = 3.5
IEEE_TWO_COL = 7.16
DIRECTIONS = ['pd_to_hd', 'hd_to_pd', 'pd_to_als', 'als_to_pd', 'hd_to_als',
               ↪ 'als_to_hd']
DIR_LABEL = {d: d.replace('_to_', '→').upper() for d in DIRECTIONS}
CLF_ORDER = ['rf', 'knn', 'svm', 'dt', 'qda', 'xgb', 'lgbm']
CLF_LABEL = {'rf': 'RF', 'knn': 'KNN', 'svm': 'SVM', 'dt': 'DT', 'qda': 'QDA',
             ↪ 'xgb': 'XGBoost', 'lgbm': 'LightGBM'}
COND_COLOR = {'pd': '#4C78A8', 'hd': '#F58518', 'als': '#54A24B'}
feature_cols = list(ALL_FEATURE_COLS)

def savefig_stem(fig: plt.Figure, stem: str) -> None:
    pdf_path = FIG_PDF / f'{stem}.pdf'
    png_path = FIG_PNG / f'{stem}.png'
    fig.savefig(pdf_path)
    fig.savefig(png_path, dpi=300)
    print(f'Saved figure: {pdf_path.relative_to(REPO_ROOT)}')
    print(f'Saved figure: {png_path.relative_to(REPO_ROOT)}')

def save_table_csv(df: pd.DataFrame, name: str, *, index: bool = False) -> None:
    path = TABLE_DIR / name
    df.to_csv(path, index=index)
    print(f'Saved table: {path.relative_to(REPO_ROOT)}')

```

```

def feature_family(feature_name: str) -> str:
    if feature_name in {'cv_stride', 'cv_swing'}:
        return 'variability'
    if feature_name == 'dfa_alpha_stride':
        return 'fractal'
    if 'asymmetry' in feature_name:
        return 'asymmetry'
    if feature_name.endswith('_pct'):
        return 'phase_percentage'
    return 'raw_timing'

def as_feature_map(values, names):
    if isinstance(values, dict):
        return {name: float(values.get(name, 0.0)) for name in names}
    return {name: float(value) for name, value in zip(names, values)}

def plot_local_waterfall(ax, shap_vector, feature_vector, names, title,
    ↪top_k=8):
    order = np.argsort(np.abs(shap_vector))[:, -1][:top_k][:, -1]
    contrib = shap_vector[order]
    labels = [f"{names[i]} = {feature_vector[i]:.3f}" for i in order]
    colors = ['#C44E52' if value > 0 else '#4C78A8' for value in contrib]
    ax.barh(np.arange(len(order)), contrib, color=colors, edgecolor='black',
    ↪linewidth=0.4)
    ax.axvline(0.0, color='black', linewidth=0.8)
    ax.set_yticks(np.arange(len(order)))
    ax.set_yticklabels(labels, fontsize=7)
    ax.set_xlabel('Stored SHAP contribution')
    ax.set_title(title)
    ax.grid(axis='x', alpha=0.2)

BASE = REPO_ROOT / 'experiments' / 'results' / 'v4'
CACHE_DIR = REPO_ROOT / 'experiments' / 'shap' / 'v4'
shap_results = json.loads((BASE / 'shap_results_v4.json').read_text())['data']
cross_results = json.loads((BASE / 'cross_condition_results_v4.json').
    ↪read_text())

raw_records = []
norm_records = []
family_records = []
for direction, payload in shap_results.items():
    for clf_name, entry in payload.items():

```

```

if clf_name == '__consensus__':
    continue
raw_map = as_feature_map(entry['delta_j'], feature_cols)
norm_map = as_feature_map(entry['delta_j_normalized'], feature_cols)
for feature_name in feature_cols:
    raw_records.append({
        'direction': direction,
        'classifier': clf_name,
        'feature': feature_name,
        'delta_j': raw_map[feature_name],
    })
    norm_records.append({
        'direction': direction,
        'classifier': clf_name,
        'feature': feature_name,
        'delta_j_norm': norm_map[feature_name],
    })
for family_name, value in entry['family_delta_j'].items():
    family_records.append({
        'direction': direction,
        'classifier': clf_name,
        'family': family_name,
        'delta_j_family': float(value['delta_j_sum']) if
↳ isinstance(value, dict) else float(value),
    })

raw_df = pd.DataFrame(raw_records)
norm_df = pd.DataFrame(norm_records)
family_df = pd.DataFrame(family_records)
save_table_csv(raw_df, 'step4_delta_j_long_v4.csv')
save_table_csv(norm_df, 'step4_delta_j_normalized_long_v4.csv')
save_table_csv(family_df, 'step4_family_delta_j_long_v4.csv')

```

Saved table: report/tables/v4/step4_delta_j_long_v4.csv

Saved table: report/tables/v4/step4_delta_j_normalized_long_v4.csv

Saved table: report/tables/v4/step4_family_delta_j_long_v4.csv

1.1 j heatmaps: raw RF shifts and normalized multi-classifier consensus

The raw RF map and the normalized consensus map are saved separately because they answer different questions. The RF heatmap gives one familiar, model-specific view of reliance movement, whereas the normalized consensus map is more paper-relevant because it highlights features that move repeatedly across classifier families. The main signal is not a biomarker claim. It is that transfer repeatedly changes reliance on **timing-structure features**, especially **variability descriptors** and **raw timing features**.

```

[2]: rf_raw = (
    raw_df[raw_df['classifier'] == 'rf']

```

```

        .pivot(index='feature', columns='direction', values='delta_j')
        .reindex(feature_cols)
    )
row_order = rf_raw.mean(axis=1).sort_values(ascending=False).index.tolist()
rf_raw = rf_raw.loc[row_order, DIRECTIONS]

norm_mean = (
    norm_df.groupby(['feature', 'direction'], as_index=False)['delta_j_norm']
    .mean()
    .pivot(index='feature', columns='direction', values='delta_j_norm')
    .reindex(row_order)
)

fig, ax = plt.subplots(figsize=(IEEE_TWO_COL * 0.88, 4.2),
    constrained_layout=True)
sns.heatmap(
    rf_raw.rename(index=str, columns=DIR_LABEL),
    cmap=sns.light_palette('#8B1E3F', as_cmap=True),
    annot=True,
    fmt='.3f',
    linewidths=0.5,
    linecolor='white',
    cbar_kws={'label': 'Raw j (RF)'},
    ax=ax,
)
ax.set_title('Raw feature-importance shift (j) for the RF classifier')
ax.set_xlabel('Transfer direction')
ax.set_ylabel('Feature')
savefig_stem(fig, 'step4_delta_j_rf_heatmap')
plt.show()

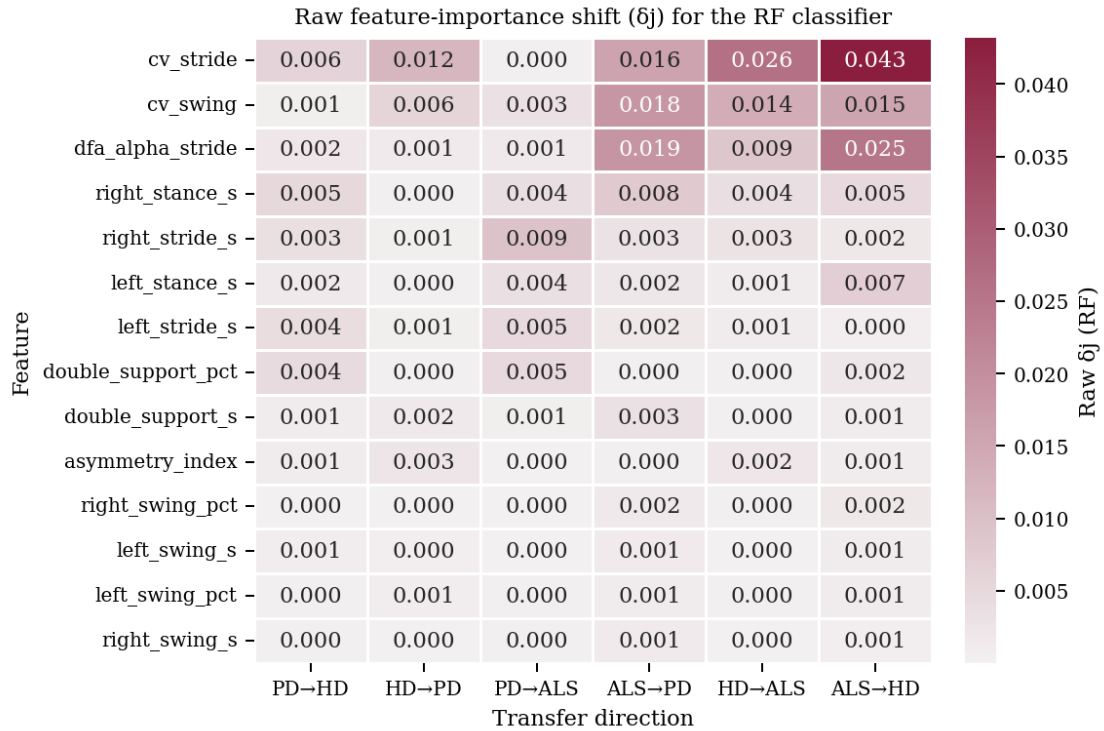
fig, ax = plt.subplots(figsize=(IEEE_TWO_COL * 0.88, 4.2),
    constrained_layout=True)
sns.heatmap(
    norm_mean.rename(index=str, columns=DIR_LABEL),
    cmap=sns.light_palette('#6C3483', as_cmap=True),
    annot=True,
    fmt='.3f',
    linewidths=0.5,
    linecolor='white',
    cbar_kws={'label': 'Mean normalized j across classifiers'},
    ax=ax,
)
ax.set_title('Normalized feature-importance shift consensus across classifier_
    families')
ax.set_xlabel('Transfer direction')
ax.set_ylabel('Feature')

```

```
savefig_stem(fig, 'step4_delta_j_normalized_consensus_heatmap')
plt.show()
```

Saved figure: report/figures/v4/pdf/step4/step4_delta_j_rf_heatmap.pdf

Saved figure: report/figures/v4/png/step4/step4_delta_j_rf_heatmap.png

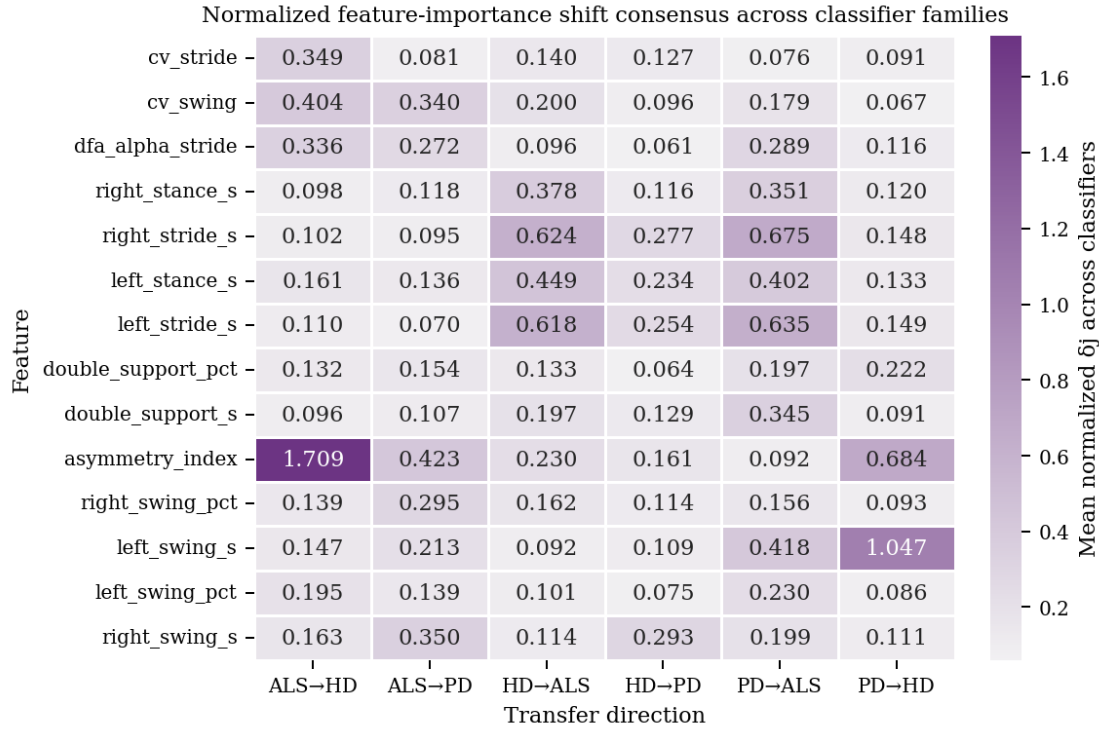


Saved figure:

report/figures/v4/pdf/step4/step4_delta_j_normalized_consensus_heatmap.pdf

Saved figure:

report/figures/v4/png/step4/step4_delta_j_normalized_consensus_heatmap.png



1.2 dfa_alpha_stride directionality and feature-family movement

`dfa_alpha_stride` remains worth isolating because it recurs across several transfer settings, but it should still be described diagnostically. At the family level, the strongest average movement is carried by **variability** (0.0302) and **raw timing** (0.0274), with **fractal** structure still visible (0.0124) and the remaining families much smaller. That hierarchy aligns with the feature-level heatmaps and strengthens the high-level Step 4 story without pushing it into biomarker language.

```
[3]: dfa_series = raw_df[(raw_df['classifier'] == 'rf') & (raw_df['feature'] ==
    ↪ 'dfa_alpha_stride')].set_index('direction')['delta_j'].reindex(DIRECTIONS)

fig, ax = plt.subplots(figsize=(IEEE_ONE_COL * 1.3, 3.1),
    ↪ constrained_layout=True)
bars = ax.bar(range(len(DIRECTIONS)), dfa_series.to_numpy(), color='#4C78A8',
    ↪ edgecolor='black', linewidth=0.5)
ax.set_xticks(range(len(DIRECTIONS)))
ax.set_xticklabels([DIR_LABEL[d] for d in DIRECTIONS], rotation=35, ha='right')
ax.set_ylabel('Raw j (RF)')
ax.set_title('Directional movement of dfa_alpha_stride under transfer')
ax.grid(axis='y', alpha=0.2)
ax.set_ylim(0.0, 0.04)
ax.set_yticks(np.linspace(0.0, 0.04, 5))
for bar, value in zip(bars, dfa_series.to_numpy()):
```

```

        label_y = min(float(value) + 0.0012, 0.039)
        ax.text(bar.get_x() + bar.get_width() / 2, label_y, f'{value:.3f}',
        ↪ha='center', va='bottom', fontsize=8)
savefig_stem(fig, 'step4_dfa_alpha_stride_directionality')
plt.show()

family_mean = (
    family_df.groupby(['family', 'direction'], as_index=False)['delta_j_family']
    .mean()
    .pivot(index='family', columns='direction', values='delta_j_family')
)
family_order = family_mean.mean(axis=1).sort_values(ascending=False).index.
    ↪tolist()
family_mean = family_mean.loc[family_order, DIRECTIONS]

fig, ax = plt.subplots(figsize=(IEEE_TWO_COL * 0.86, 3.05),
    ↪constrained_layout=True)
sns.heatmap(
    family_mean.rename(columns=DIR_LABEL),
    cmap=sns.light_palette('#0B3C5D', as_cmap=True),
    annot=True,
    fmt='.3f',
    annot_kws={'size': 7},
    linewidths=0.5,
    linecolor='white',
    cbar_kws={'label': 'Mean family-level j'},
    ax=ax,
)
ax.set_title('Feature-family consensus shift across transfer directions')
ax.set_xlabel('Transfer direction')
ax.set_ylabel('Feature family')
ax.tick_params(axis='x', labels=7, rotation=20)
ax.tick_params(axis='y', labels=7)
savefig_stem(fig, 'step4_family_delta_j_consensus')
plt.show()

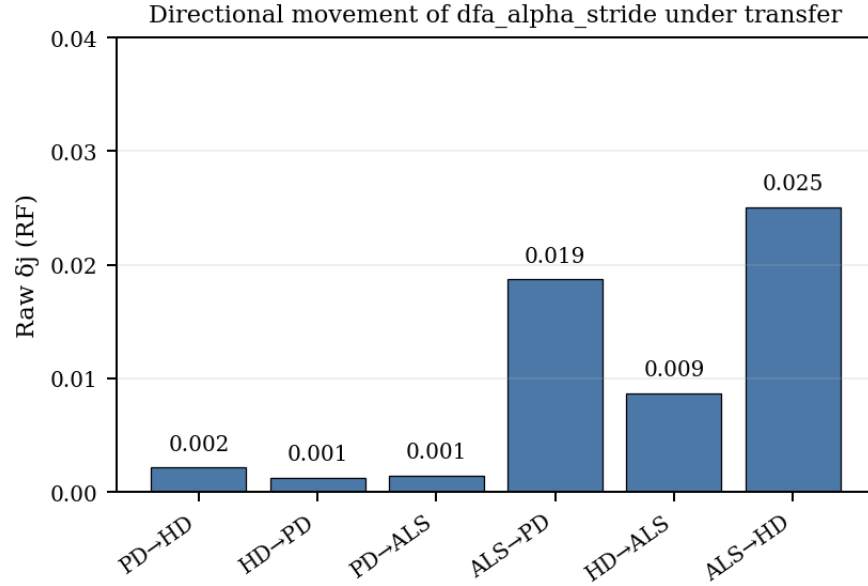
```

Saved figure:

report/figures/v4/pdf/step4/step4_dfa_alpha_stride_directionality.pdf

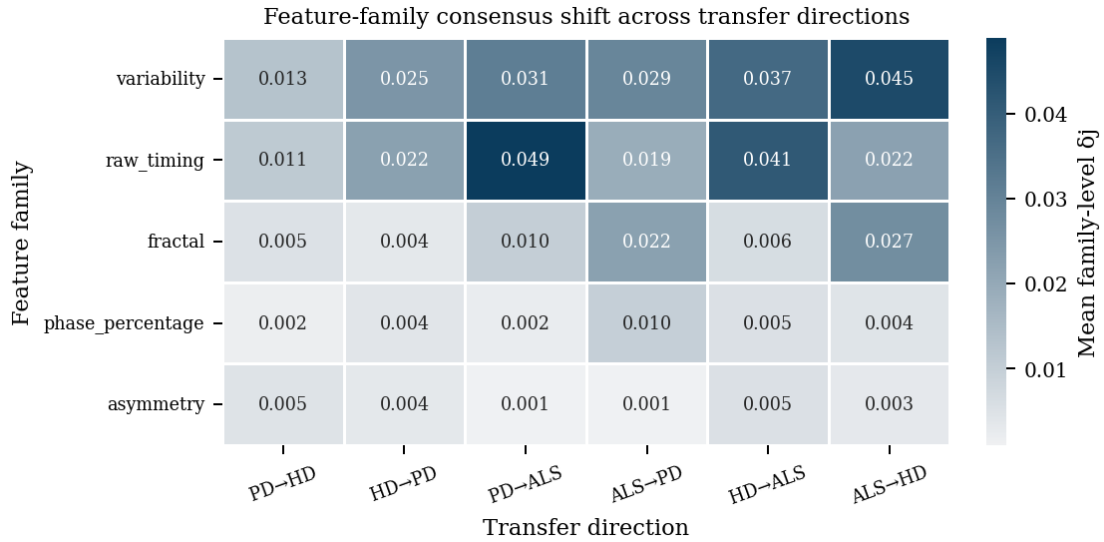
Saved figure:

report/figures/v4/png/step4/step4_dfa_alpha_stride_directionality.png



Saved figure: report/figures/v4/pdf/step4/step4_family_delta_j_consensus.pdf

Saved figure: report/figures/v4/png/step4/step4_family_delta_j_consensus.png



1.3 Cross-classifier agreement on shifting features

The direction-wise Spearman matrices remain valuable because they reveal when different classifier families diagnose the same transfer difficulty versus different ones. Strong agreement supports a more stable diagnostic story, whereas weaker agreement warns us not to overgeneralize a single model family's SHAP explanation to the entire transfer surface.

```

[4]: fig, axes = plt.subplots(3, 2, figsize=(IEEE_TWO_COL * 1.02, 8.85),
    ↪gridspec_kw={'hspace': 0.52, 'wspace': 0.34})
axes_flat = axes.ravel()
last_mesh = None

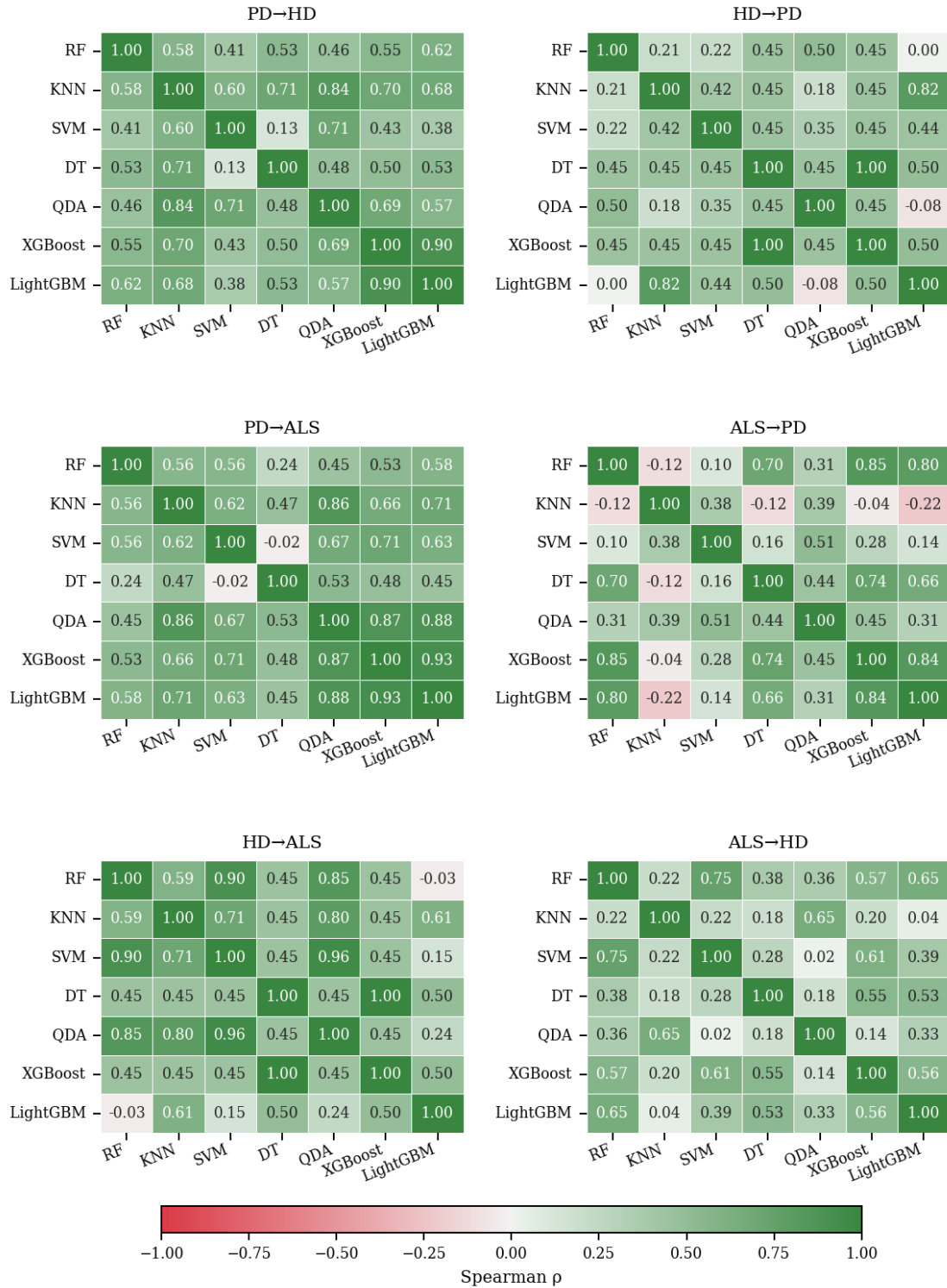
for ax, direction in zip(axes_flat, DIRECTIONS):
    block = (
        raw_df[raw_df['direction'] == direction]
        .pivot(index='feature', columns='classifier', values='delta_j')
        .reindex(feature_cols)
    )
    corr_mat = np.zeros((len(CLF_ORDER), len(CLF_ORDER)))
    for i, clf_i in enumerate(CLF_ORDER):
        for j, clf_j in enumerate(CLF_ORDER):
            rho, _ = stats.spearmanr(block[clf_i], block[clf_j])
            corr_mat[i, j] = rho
    heat = sns.heatmap(
        pd.DataFrame(corr_mat, index=[CLF_LABEL[c] for c in CLF_ORDER],
    ↪columns=[CLF_LABEL[c] for c in CLF_ORDER]),
        cmap=sns.diverging_palette(10, 130, as_cmap=True),
        vmin=-1.0,
        vmax=1.0,
        center=0.0,
        annot=True,
        fmt='.2f',
        annot_kws={'size': 7},
        linewidths=0.4,
        linecolor='white',
        cbar=False,
        ax=ax,
    )
    last_mesh = heat.collections[0]
    ax.set_title(DIR_LABEL[direction], fontsize=8.5, pad=6)
    ax.set_xlabel('')
    ax.set_ylabel('')
    ax.tick_params(axis='x', labelrotation=22, labelsz=7, pad=1)
    ax.tick_params(axis='y', labelsz=7)
    for label in ax.get_xticklabels():
        label.set_ha('right')

fig.subplots_adjust(top=0.95, bottom=0.12)
cbar_ax = fig.add_axes([0.18, 0.045, 0.64, 0.02])
cbar = fig.colorbar(last_mesh, cax=cbar_ax, orientation='horizontal')
cbar.set_label('Spearman ', fontsize=8)
cbar.ax.tick_params(labelsz=7)
savefig_stem(fig, 'step4_delta_j_spearman')
plt.show()

```

Saved figure: report/figures/v4/pdf/step4/step4_delta_j_spearman.pdf

Saved figure: report/figures/v4/png/step4/step4_delta_j_spearman.png



1.4 Within-vs-cross SHAP importance profiles and top-feature recurrence

These plots are retained because they show whether transfer changes feature reliance systematically rather than idiosyncratically. The recurrence summary is especially concrete: **cv_stride** appears **33** times in the top-3 shift lists, **cv_swing** appears **21** times, and **dfa_alpha_stride** appears **19** times. Those feature-level recurrences are consistent with the family-level result that **variability** and **raw timing** dominate the aggregate movement story.

```
[5]: fig, axes = plt.subplots(2, 3, figsize=(IEEE_TWO_COL, 7.0),
    ↪ constrained_layout=True)
    axes_flat = axes.ravel()
    for ax, direction in zip(axes_flat, DIRECTIONS):
        entry = shap_results[direction]['rf']
        within = np.asarray(entry['mean_abs_within'], dtype=float)
        cross = np.asarray(entry['mean_abs_cross'], dtype=float)
        order = np.argsort(within)
        names = [feature_cols[i] for i in order]
        y = np.arange(len(names))
        ax.barh(y - 0.18, within[order], height=0.36, color='#4C78A8',
    ↪ label='Within', edgecolor='black', linewidth=0.3)
        ax.barh(y + 0.18, cross[order], height=0.36, color='#C44E52',
    ↪ label='Cross', edgecolor='black', linewidth=0.3)
        ax.set_yticks(y)
        ax.set_yticklabels(names, fontsize=6.5)
        ax.set_title(f"{DIR_LABEL[direction]} (RF)")
        ax.grid(axis='x', alpha=0.18)
    axes_flat[0].legend(loc='lower right', frameon=False)
    savefig_stem(fig, 'step4_rf_within_vs_cross_importance')
    plt.show()

    recurrence_rows = []
    for direction in DIRECTIONS:
        for clf_name in CLF_ORDER:
            block = raw_df[(raw_df['direction'] == direction) &
    ↪ (raw_df['classifier'] == clf_name)].sort_values('delta_j', ascending=False).
    ↪ head(3)
            for rank, row in enumerate(block.itertuples(), start=1):
                recurrence_rows.append({'direction': direction, 'classifier':
    ↪ clf_name, 'feature': row.feature, 'rank': rank, 'delta_j': row.delta_j})
    recurrence_df = pd.DataFrame(recurrence_rows)
    recurrence_counts = recurrence_df.groupby('feature', as_index=False).size().
    ↪ rename(columns={'size': 'top3_count'}).sort_values('top3_count',
    ↪ ascending=False)
    save_table_csv(recurrence_counts, 'step4_top_feature_recurrence_v4.csv')

    display(
        recurrence_df.head(24).style
```

```

        .format({'delta_j': '{:.4f}'})
        .set_caption('Top-three shifted features by direction and classifier')
    )

fig, ax = plt.subplots(figsize=(IEEE_ONE_COL * 1.25, 3.1),
    constrained_layout=True)
plot_df = recurrence_counts.head(12).iloc[::-1]
ax.barh(plot_df['feature'], plot_df['top3_count'], color='#6C3483',
    edgecolor='black', linewidth=0.4)
ax.set_xlabel('Number of appearances in the top-3 j set')
ax.set_title('Recurring transfer-shift features across classifier families')
ax.grid(axis='x', alpha=0.2)
for idx, value in enumerate(plot_df['top3_count']):
    ax.text(value + 0.08, idx, str(int(value)), va='center', fontsize=8)
savefig_stem(fig, 'step4_top_feature_recurrence')
plt.show()

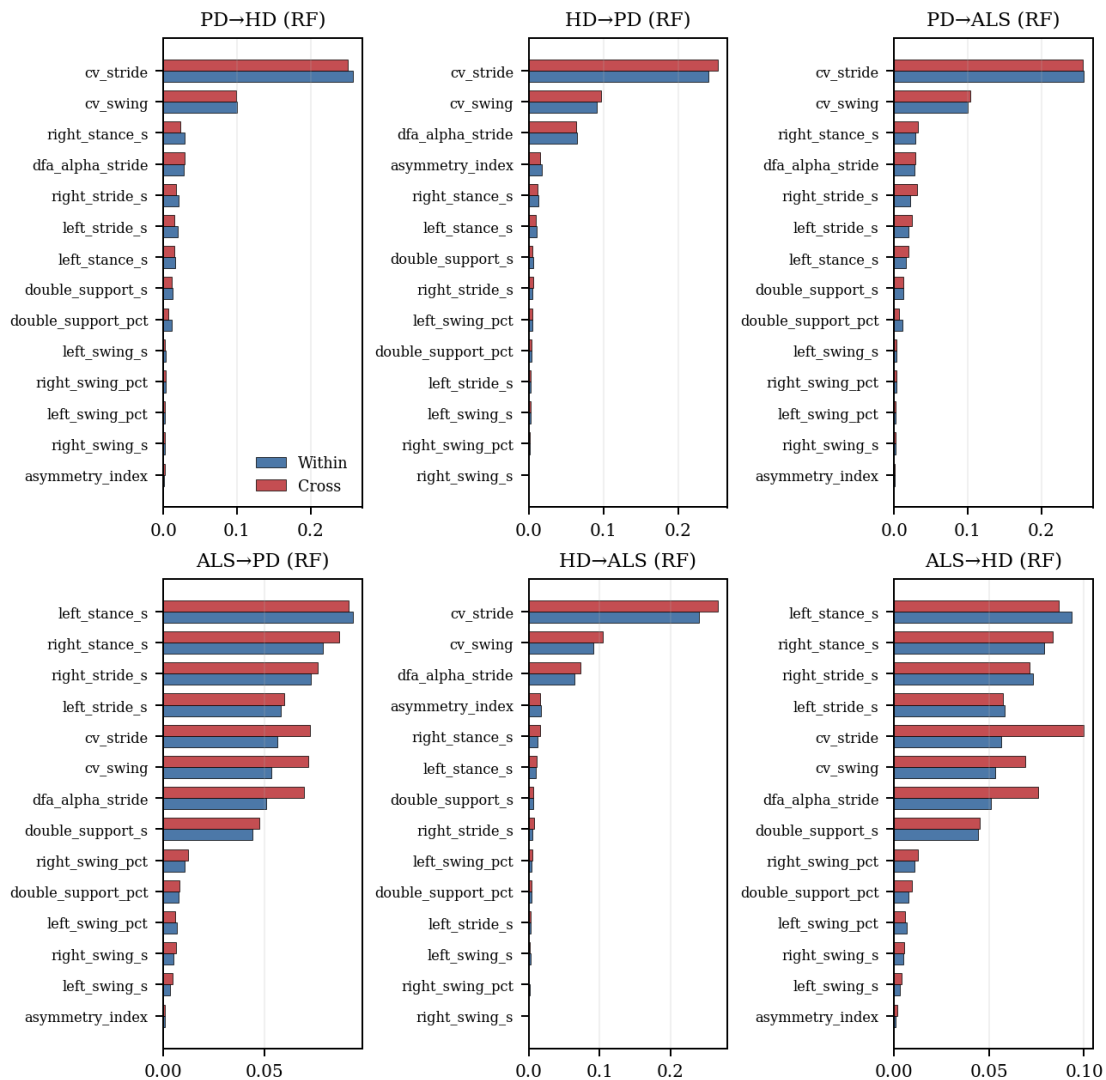
```

Saved figure:

report/figures/v4/pdf/step4/step4_rf_within_vs_cross_importance.pdf

Saved figure:

report/figures/v4/png/step4/step4_rf_within_vs_cross_importance.png

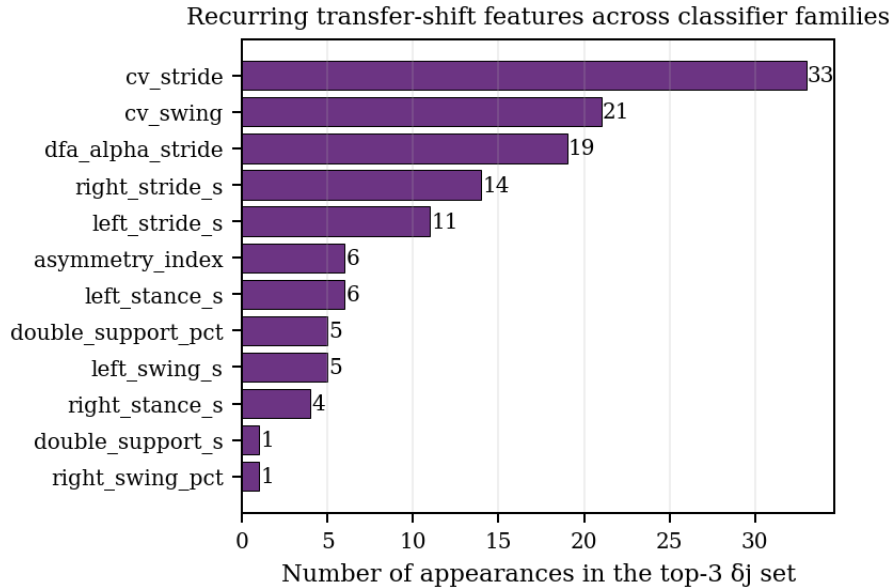


Saved table: report/tables/v4/step4_top_feature_recurrence_v4.csv

<pandas.io.formats.style.Styler at 0x7f2950dc0560>

Saved figure: report/figures/v4/pdf/step4/step4_top_feature_recurrence.pdf

Saved figure: report/figures/v4/png/step4/step4_top_feature_recurrence.png



1.5 Direction-level scatter, dependence-style views, and representative local explanations

The scatter, dependence-style plots, and local waterfall examples are supplement-grade diagnostics. They are helpful when a specific transfer direction needs closer inspection, but they should remain subordinate to the family-level and recurrence-level summaries in the main paper. The caveat must stay strong in the notebook text: these are **diagnostic attribution views**, not causal evidence of disease-specific gait biomarkers.

```
[6]: scatter_rows = []
for direction in DIRECTIONS:
    source, _ = direction.split('_to_')
    for clf_name in CLF_ORDER:
        entry = shap_results[direction][clf_name]
        scatter_rows.append({
            'direction': direction,
            'source': source,
            'classifier': clf_name,
            'max_delta_j_norm': float(np.max(np.
↳asarray(entry['delta_j_normalized'], dtype=float))),
            'matched_degradation_subject':
↳float(cross_results[direction]['classifiers'][clf_name]['delta_f1_subject']),
        })
scatter_df = pd.DataFrame(scatter_rows)
save_table_csv(scatter_df, 'step4_delta_j_vs_degradation_v4.csv')
```

```

fig, ax = plt.subplots(figsize=(IEEE_ONE_COL * 1.55, 3.15),
    ↪constrained_layout=True)
source_handles = []
for source, color in COND_COLOR.items():
    frame = scatter_df[scatter_df['source'] == source]
    ax.scatter(frame['max_delta_j_norm'], frame['matched_degradation_subject'],
    ↪s=42, alpha=0.8, color=color, edgecolors='white', linewidths=0.5)
    source_handles.append(plt.Line2D([0], [0], marker='o', color='none',
    ↪markerfacecolor=color, markeredgecolor='white', markersize=7,
    ↪label=f"{source.upper()} source"))
if len(scatter_df) >= 3:
    slope, intercept = np.polyfit(scatter_df['max_delta_j_norm'],
    ↪scatter_df['matched_degradation_subject'], 1)
    x_line = np.linspace(scatter_df['max_delta_j_norm'].min(),
    ↪scatter_df['max_delta_j_norm'].max(), 200)
    ax.plot(x_line, slope * x_line + intercept, color='black', linestyle=':',
    ↪linewidth=1.1)
rho, _ = stats.spearmanr(scatter_df['max_delta_j_norm'],
    ↪scatter_df['matched_degradation_subject'])
outliers = scatter_df.nlargest(2, 'max_delta_j_norm')
for row in outliers.itertuples():
    ax.annotate(
        f"{DIR_LABEL[row.direction]} / {CLF_LABEL[row.classifier]}",
        (row.max_delta_j_norm, row.matched_degradation_subject),
        textcoords='offset points',
        xytext=(5, 5),
        fontsize=7,
        color='#2C3E50',
    )
ax.set_xlabel('Largest normalized j within the direction/classifier pair')
ax.set_ylabel('Matched degradation\n(within - cross)', fontsize=8.2)
ax.set_title(f'Descriptive link between SHAP shift and transfer degradation
    ↪(Spearman = {rho:.2f})')
ax.grid(alpha=0.2)
trend_handle = plt.Line2D([0], [0], color='black', linestyle=':', linewidth=1.
    ↪1, label='Least-squares trend')
ax.legend(handles=source_handles + [trend_handle], loc='center left',
    ↪bbox_to_anchor=(1.02, 0.5), frameon=False)
savefig_stem(fig, 'delta_j_vs_degradation')
plt.show()

npz = np.load(CACHE_DIR / 'pd_rf_cross_hd.npz')
feature_matrix = np.asarray(npz['X_explained'], dtype=float)
shap_values = np.asarray(npz['shap_values'], dtype=float)
y_true = np.asarray(cross_results['pd_to_hd']['classifiers']['rf']['y_true'],
    ↪dtype=int)

```



```

y_pred = np.asarray(cross_results['pd_to_hd']['classifiers']['rf']['y_pred'],
    dtype=int)
y_prob = np.asarray(cross_results['pd_to_hd']['classifiers']['rf']['y_prob'],
    dtype=float)
assert len(feature_matrix) == len(shap_values) == len(y_true) == len(y_pred) ==
    len(y_prob)

feature_subset = ['cv_stride', 'cv_swing', 'dfa_alpha_stride']
feature_index = [feature_cols.index(name) for name in feature_subset]
fig, axes = plt.subplots(1, 3, figsize=(IEEE_TWO_COL, 2.7),
    constrained_layout=True)
for ax, feat_name, feat_idx in zip(axes, feature_subset, feature_index):
    ax.scatter(feature_matrix[:, feat_idx], shap_values[:, feat_idx], s=9,
        alpha=0.18, color='#4C78A8')
    rho_feat, _ = stats.spearmanr(feature_matrix[:, feat_idx], shap_values[:,
        feat_idx])
    ax.set_title(f"{feat_name} (={rho_feat:.2f})")
    ax.set_xlabel('Feature value')
    ax.set_ylabel('Stored SHAP value')
    ax.grid(alpha=0.18)
savefig_stem(fig, 'step4_dependence_proxies')
plt.show()

misclassified = np.where(y_true != y_pred)[0]
if len(misclassified) == 0:
    print('No misclassified PD→HD RF strides were found; standalone waterfall
        figures were skipped.')
else:
    confidence = np.abs(y_prob[misclassified] - 0.5)
    top_indices = misclassified[np.argsort(confidence)[::-1][:min(3,
        len(misclassified))]]
    for plot_idx, stride_idx in enumerate(top_indices, start=1):
        fig, ax = plt.subplots(figsize=(IEEE_ONE_COL * 1.25, 2.8),
            constrained_layout=True)
        title = (
            f"PD→HD RF local explanation
            #{plot_idx}\ntrue={int(y_true[stride_idx])}, pred={int(y_pred[stride_idx])},
            P(disease)={y_prob[stride_idx]:.3f}"
        )
        plot_local_waterfall(ax, shap_values[stride_idx],
            feature_matrix[stride_idx], feature_cols, title)
        savefig_stem(fig, f'waterfall_pd_to_hd_case_{plot_idx}')
        plt.show()

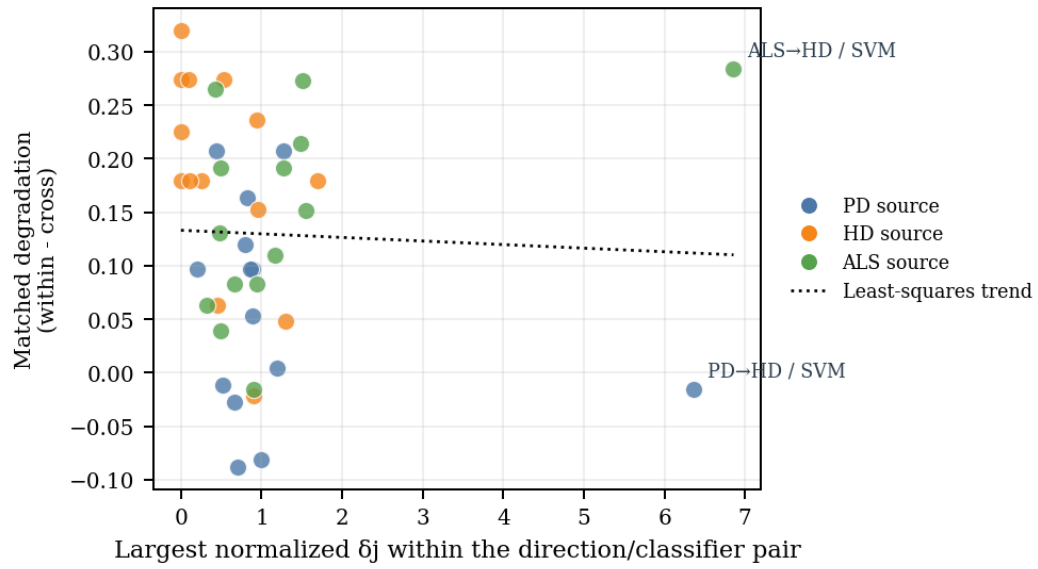
```

Saved table: report/tables/v4/step4_delta_j_vs_degradation_v4.csv

Saved figure: report/figures/v4/pdf/step4/delta_j_vs_degradation.pdf

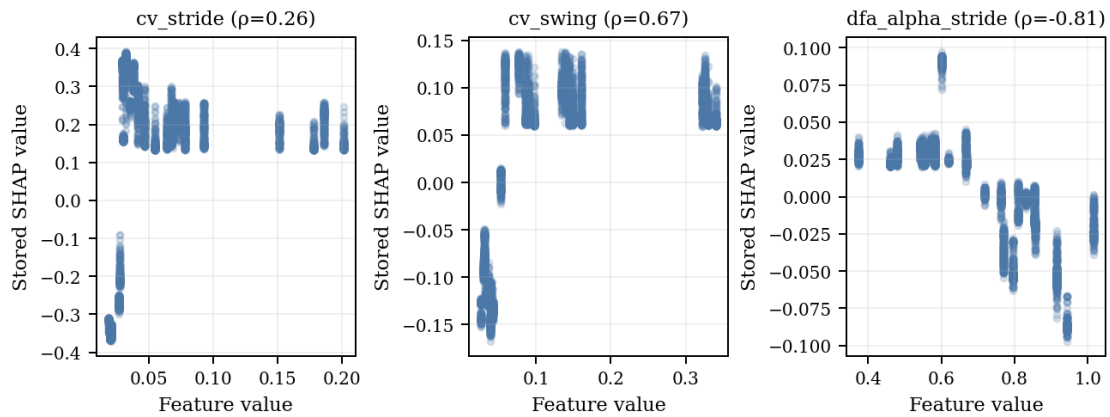
Saved figure: report/figures/v4/png/step4/delta_j_vs_degradation.png

Descriptive link between SHAP shift and transfer degradation (Spearman $\rho=-0.20$)



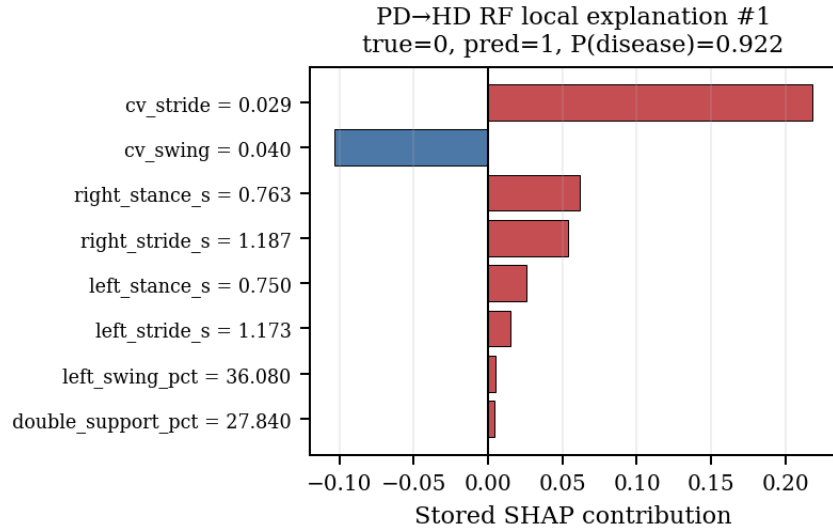
Saved figure: report/figures/v4/pdf/step4/step4_dependence_proxies.pdf

Saved figure: report/figures/v4/png/step4/step4_dependence_proxies.png

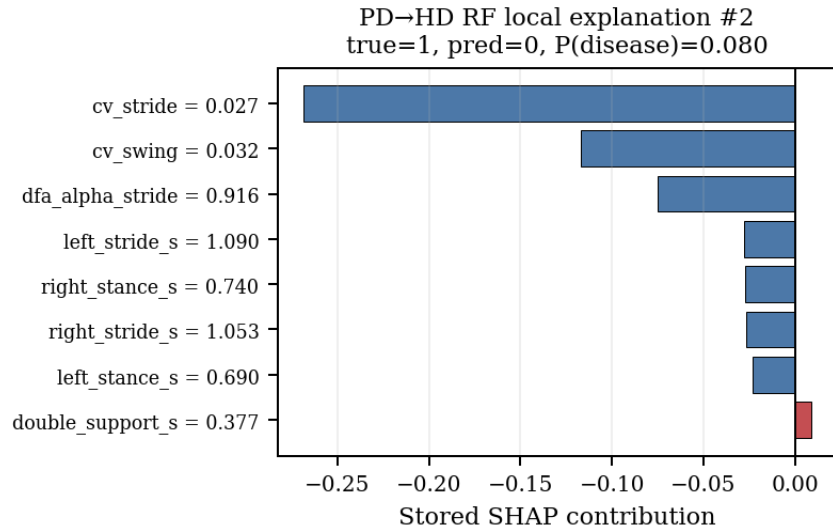


Saved figure: report/figures/v4/pdf/step4/waterfall_pd_to_hd_case_1.pdf

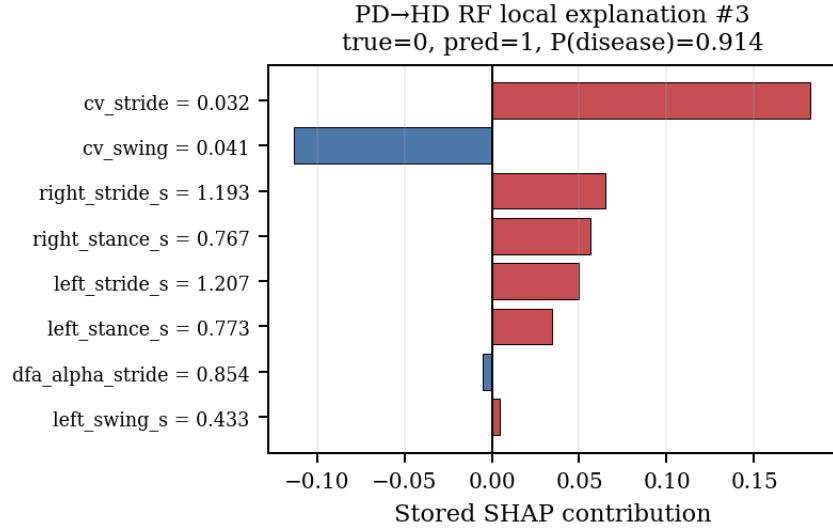
Saved figure: report/figures/v4/png/step4/waterfall_pd_to_hd_case_1.png



Saved figure: report/figures/v4/pdf/step4/waterfall_pd_to_hd_case_2.pdf
Saved figure: report/figures/v4/png/step4/waterfall_pd_to_hd_case_2.png



Saved figure: report/figures/v4/pdf/step4/waterfall_pd_to_hd_case_3.pdf
Saved figure: report/figures/v4/png/step4/waterfall_pd_to_hd_case_3.png



1.6 Interpretation notes

Large `_j` values indicate that a transferred model relied on a feature differently than its within-source counterpart; they do **not** establish mechanism. The repaired v4 Step 4 setup matters because the SHAP backgrounds are source-specific and avoid the invalid cross-direction reuse problem from earlier lines. The most consistent high-level Step 4 signal is therefore modest but meaningful: transfer perturbs reliance most clearly in **variability**, **raw timing**, and a small set of recurring timing-structure descriptors.